

SKILL WEEK8

TASK 1

To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_LINE 500

typedef struct {
    char name[50];
    char value[100];
} Variable;

Variable variables[] = {
    {"USER", "Laasya"},
    {"HOME", "/home/student"},
    {"CITY", "Hyderabad"},
    {"COURSE", "CSE"}
};

int variable_count = 4;

/* Find variable value */
const char *get_value(const char *name)
{
    for (int i = 0; i < variable_count; i++) {
        if (strcmp(variables[i].name, name) == 0)
            return variables[i].value;
    }

    return NULL;
}

/* Expand variables such as $USER */
void expand_variables(char *input, char *output)
{
    int i = 0, j = 0;
```

```

while (input[i] != '\0') {

    if (input[i] == '$') {

        i++;

        /* Handle ${VARIABLE} */
        if (input[i] == '{') {
            i++;

            char name[50];
            int k = 0;

            while (input[i] != '}' &&
                    input[i] != '\0' &&
                    k < 49) {
                name[k++] = input[i++];
            }

            name[k] = '\0';

            if (input[i] == '}')
                i++;

            const char *value = get_value(name);

            if (value != NULL) {
                strcpy(&output[j], value);
                j += strlen(value);
            } else {
                printf("Undefined variable: %s\n", name);
            }
        }

        /* Handle $VARIABLE */
        else {

```

```

    }
}

output[j] = '\0';
}

int main()
{
    char input[MAX_LINE];
    char output[MAX_LINE * 2];

    printf("==== VARIABLE EXPANSION TEST =====\n");

    printf("Available variables:\n");
    for (int i = 0; i < variable_count; i++) {
        printf("%s = %s\n",
            variables[i].name,
            variables[i].value);
    }

    printf("\nEnter a string containing variables:\n");
    printf("> ");

    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    expand_variables(input, output);

    printf("\nOriginal token : %s\n", input);
    printf("Expanded token : %s\n", output);

    return 0;
}

```

```
laasya@laasya: ~/skill_week8 × + v
laasya@laasya:~$ mkdir skill_week8
laasya@laasya:~$ cd skill_week8
laasya@laasya:~/skill_week8$ nano task1.c
laasya@laasya:~/skill_week8$ gcc task1.c -o task1
laasya@laasya:~/skill_week8$ ./task1
===== VARIABLE EXPANSION TEST =====
Available variables:
USER = Laasya
HOME = /home/student
CITY = Hyderabad
COURSE = CSE

Enter a string containing variables:
> hello USER from $CITY

Original token : hello USER from $CITY
Expanded token : hello USER from Hyderabad
laasya@laasya:~/skill_week8$
```

TASK 2

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_INPUT 100

/* Built-in command functions */

void builtin_pwd()
{
    char cwd[500];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("Current directory: %s\n", cwd);
    else
        perror("pwd");
}

void builtin_echo(char *args)
{
    printf("%s\n", args);
}

void builtin_help()
{
    printf("\nAvailable built-in commands:\n");
    printf("  pwd    - Display current directory\n");
    printf("  echo   - Display text\n");
    printf("  cd     - Change directory\n");
    printf("  help   - Display available commands\n");
    printf("  exit   - Exit the program\n");
}

void builtin_cd(char *path)
{
    if (path == NULL || strlen(path) == 0) {
```

```

    if (path == NULL || strlen(path) == 0) {
        printf("cd: missing directory\n");
        return;
    }

    if (chdir(path) != 0)
        perror("cd");
}

/* Dispatch table */

typedef void (*CommandFunction)(char *);

typedef struct {
    char *name;
    CommandFunction function;
} Command;

/* Wrapper functions for dispatch table */

void execute_pwd(char *args)
{
    (void)args;
    builtin_pwd();
}

void execute_help(char *args)
{
    (void)args;
    builtin_help();
}

void execute_cd(char *args)
{
    builtin_cd(args);
}

```

```

printf("\nmyshell> ");

if (fgets(input, sizeof(input), stdin) == NULL)
    break;

input[strcspn(input, "\n")] = '\0';

if (strlen(input) == 0)
    continue;

/* Exit command */
if (strcmp(input, "exit") == 0) {
    printf("Exiting shell...\n");
    break;
}

/* Separate command and arguments */

char *command = strtok(input, " ");
char *args = strtok(NULL, "");

if (command == NULL)
    continue;

/* Dispatch command */

if (!dispatch_command(command, args)) {
    printf("Invalid command: %s\n", command);
    printf("Type 'help' for available commands.\n");
}

}

return 0;
}

```

```

laasya@laasya:~/skill_week8$ nano task2.c
laasya@laasya:~/skill_week8$ gcc task2.c -o task2
laasya@laasya:~/skill_week8$ ./task2
===== BUILT-IN COMMAND DISPATCHER =====
Type 'help' to see available commands.

```

```
myshell> help
```

Available built-in commands:

```

pwd      - Display current directory
echo     - Display text
cd       - Change directory
help     - Display available commands
exit     - Exit the program

```

```
myshell> |
```